

PROJETO INTEGRADOR - 6O PERIODO - ENVIO FINAL PARA BANCA

Game Hub

Documento de Especificacao Final

Versao consolidada para apresentacao, considerando os feedbacks recebidos na Fase 02. O documento demonstra, com base no codigo real do projeto, a integracao entre frontend React/Inertia e backend Laravel, incluindo rotas, services, payloads, regras de negocio, testes e build.

Versao do documento 1.5 - Versao final para banca	Data da revisao 15/05/2026
Aplicacao analisada Laravel 12 + React 19 + Inertia.js	Banco local SQLite em <code>database/database.sqlite</code>

Equipe

Murilo Cesar Ramos Melo - 2310194
Marcelo Alencar Quessada - 2321520
Otavio - 2320135

Parte 00 - Historico de Versao do Projeto

Versao	Marco	Responsavel	Registro da alteracao
1.0	Fase 01	Grupo	Documento inicial, definicao do produto, requisitos e arquitetura prevista.
1.2	6o periodo - Fase 01	Grupo	Registro do backend em desenvolvimento, regras de negocio, modelo de dados e requisitos planejados.
1.4	Revisao da Fase 02	Grupo	Documento corrigido com evidencias de integracao frontend/backend, payloads, services, testes e build.
1.5	Envio final para banca	Grupo	Consolidacao final no formato PDF, com requisitos da Fase 01 comparados ao estado real da Fase 02.

Parte 01 - Identificacao do Produto, Equipe e Repositorio

Produto

O Game Hub e uma aplicacao web voltada ao ecossistema de games independentes. O objetivo e permitir que desenvolvedores, artistas e profissionais da area mantenham um perfil, publiquem portfolio, descubram outros usuarios e realizem interacoes sociais basicas como seguir, deixar de seguir, bloquear e desbloquear perfis.

Equipe atual

Integrante	Matricula	Participacao no projeto
Murilo Cesar Ramos Melo	2310194	Implementacao, revisao da integracao, testes e consolidacao da entrega.
Marcelo Alencar Quessada	2321520	Apoio na documentacao, revisao de requisitos e validacao funcional.
Otavio	2320135	Apoio no projeto e na organizacao da entrega final.

Artefatos

Artefato	Local	Situacao
Repositorio Git	https://github.com/LucasJRamos/teste_gamehub.git	Repositorio oficial do projeto, com a branch murilo contendo a versao final enviada para banca.
Documento final	DOCUMENTO DE ESPECIFICACAO GAME HUB - VERSAO FINAL BANCA.pdf	PDF final para envio da Fase 03.

Parte 02 - Planejamento do Semestre

Atividade tecnica	Status final	Evidencia
Autenticacao integrada	Concluida	AuthController, AuthService, Login.jsx, Register.jsx.
Dashboard autenticado	Concluido	DashboardController entrega usuario atual e sugestoes para Dashboard/Index.jsx.
Perfil e portfolio	Concluido	ProfileController, ProfileService, PortfolioManager.jsx.
Busca de usuarios	Concluida	UserDirectoryService filtra por nome/titulo e remove usuarios bloqueados da busca comum.
Follow, block e unblock	Concluido	SocialGraphService, FollowController, BlockController, UserActions.jsx.
Testes e build	Concluidos	php artisan test e npm run build executados com sucesso.

Parte 03 - Requisitos Planejados e Estado Atual

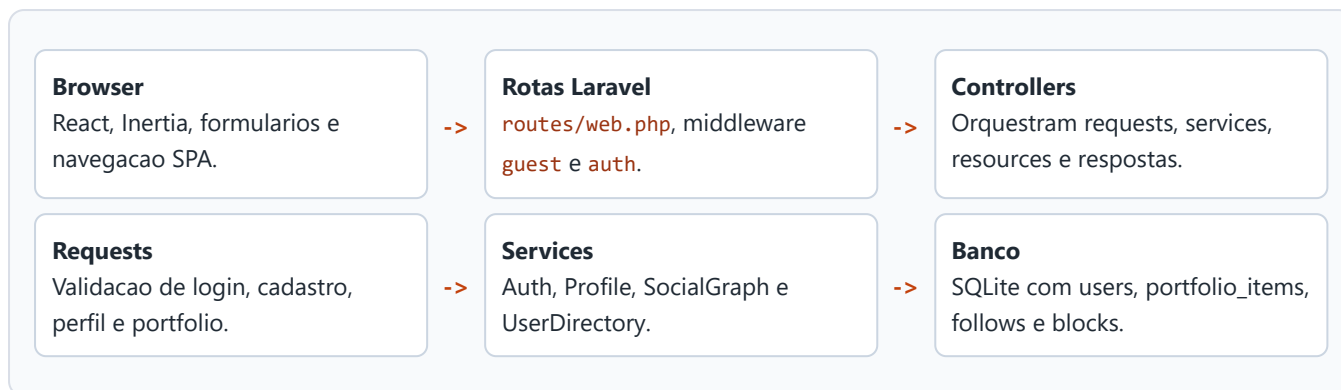
A tabela abaixo recupera os requisitos registrados no PDF da Fase 01 e compara com o que existe no codigo ao final da Fase 02. O objetivo e demonstrar evolucao sem afirmar que modulos futuros ja estao prontos.

Codigo	Requisito planejado	Status final	Evidencia atual
RF01	Cadastro de usuarios	Implementado e integrado	<code>POST /register</code> , <code>RegisterRequest</code> , <code>AuthService</code> .
RF02	Autenticacao de usuarios	Implementado e integrado	<code>POST /login</code> , sessao Laravel e redirect para dashboard.
RF03	Edicao de perfil	Implementado e integrado	<code>PUT /profile</code> , upload opcional e validacao.
RF04	Sistema de seguidores	Implementado e integrado	<code>POST/DELETE /users/{user}/follow</code> e tabela <code>follows</code> .
RF05	Bloqueio de usuarios	Implementado e integrado	<code>POST/DELETE /users/{user}/block</code> , tabela <code>blocks</code> e lista de bloqueados.
RF06	Busca de usuarios	Implementado e integrado	<code>GET /users?search=...</code> , paginacao e filtro de bloqueio.
RF07	Visualizacao de perfis publicos	Implementado e integrado	<code>GET /users/{user}</code> , <code>Profiles/Show.jsx</code> e regra de invisibilidade.
RF08	Configuracoes da conta	Parcial	Perfil permite alterar nome, e-mail, data de nascimento, titulo e foto; ainda nao ha tela separada de preferencias.
RF09	Feed de conteudo	Parcial	Portfolio publica links e imagens; ainda nao ha feed geral.
RF10	Comunidades	Planejado	Sem model, migration ou tela nesta versao.
RF11	Eventos	Planejado	Sem implementacao nesta versao.
RF12	Chat entre usuarios	Planejado	Sem implementacao nesta versao.
RF13	Marketplace de servicos	Planejado	Sem implementacao nesta versao.

Leitura da evolucao: os requisitos centrais para demonstrar integracao foram implementados. Os modulos maiores, como chat, eventos, comunidades e marketplace, permanecem como backlog para fases futuras.

20. Arquitetura Geral do Sistema

A arquitetura final usa Laravel como backend e React com Inertia.js como frontend SPA. O navegador envia acoes para rotas Laravel; o backend valida dados, executa regras nos services, persiste no SQLite via Eloquent e retorna props Inertia ou JSON quando a requisicao pede `Accept: application/json`.



Evolucao arquitetural desde a Fase 01

Ponto	Fase 01	Versao final	Ganho tecnico
Estado do sistema	Backend principal em implementacao.	Frontend e backend conectados nos fluxos principais.	O usuario executa operacoes reais pela interface.
Arquitetura	Descricao conceitual em camadas.	Camadas mapeadas para arquivos reais do repositorio.	Mais coerencia entre documento e codigo.
Regras sociais	Previstas em texto.	Implementadas em <code>SocialGraphService</code> e testadas.	Regras verificaveis por endpoint e teste.
Banco	Documento citava MySQL como previsao.	Projeto local usa SQLite conforme <code>.env</code> .	Documento ajustado ao ambiente real de avaliacao.

21. Tecnologias e Ferramentas do Projeto

Camada	Tecnologia	Onde aparece	Uso real
Backend	PHP + Laravel 12	<code>composer.json</code>	Rotas, controllers, requests, ORM, sessao e testes.
Frontend	React 19	<code>resources/js/Pages</code>	Telas e componentes da experiencia SPA.
Integracao	Inertia.js	<code>resources/js/app.js</code> , <code>HandleInertiaRequests.php</code>	Comunica React e Laravel usando sessao e props.
Build	Vite	<code>vite.config.js</code>	Compila os assets finais em <code>public/build</code> .
Banco local	SQLite	<code>.env</code>	Persistencia local sem servidor externo.
Testes	PHPUnit/Laravel Test	<code>tests/Feature</code>	Validacao de autenticao, busca, perfil, follow e block.

22. Implementacao por Camadas do Sistema

22.1 Frontend

Tela/componente	Funcionalidade	Integracao real
Auth/Login.jsx	Login	Envia POST /login e mostra erros de validacao.
Auth/Register.jsx	Cadastro	Envia POST /register , autentica e redireciona.
Dashboard/Index.jsx	Dashboard	Recebe usuario atual e sugestoes do backend.
Profiles/Edit.jsx	Editar perfil	Envia PUT /profile com forceFormData .
PortfolioManager.jsx	Portfolio	Cria e remove links/imagens por rotas Laravel.
Users/Index.jsx	Busca	Busca dinamica por GET /users com debounce.
UserActions.jsx	Follow/block/unblock	Usa links vindos do UserResource .

22.2 Backend

Endpoint	Controller	Regra aplicada
POST /register	AuthController	Valida dados, cria usuario, autentica sessao.
POST /login	AuthController	Valida credenciais e regenera sessao.
GET /dashboard	DashboardController	Carrega usuario e sugestoes.
GET /profile	ProfileController	Carrega perfil, portfolio e sugestoes.
PUT /profile	ProfileController	Valida e atualiza dados/foto.
POST /portfolio/upload	ProfileController	Cria item de portfolio link ou imagem.
GET /users	UserDirectoryController	Busca usuarios visiveis e lista bloqueados.
POST/DELETE /users/{user}/follow	FollowController	Segue/deixa de seguir, impedindo autoacao.
POST/DELETE /users/{user}/block	BlockController	Bloqueia/desbloqueia e remove follows entre usuarios.

22.3 Integracao Frontend e Backend

A comunicacao principal ocorre por Inertia. O frontend envia formularios e acoes para rotas Laravel; o backend valida, executa services, persiste no banco e devolve props para a tela. Para demonstracao tecnica, rotas criticas tambem retornam JSON quando chamadas com **Accept: application/json**.

Exemplo: login JSON

```
POST /login
Accept: application/json

{
  "email": "murilo@example.com",
  "password": "123456"
}
```

```
200 OK
{
  "message": "Login realizado com sucesso!",
  "data": {
    "id": 1,
    "username": "murilodev",
    "email": "murilo@example.com",
    "links": {
      "profile": "/profile",
      "follow": "/users/1/follow",
      "block": "/users/1/block"
    }
  }
}
```

Exemplo: busca com bloqueio aplicado

```
GET /users?search=dev
Accept: application/json

200 OK
{
  "message": "Usuarios carregados com sucesso.",
  "data": [
    { "username": "visible-dev", "has_blocked": false }
  ],
  "blocked_users": [
    { "username": "blocked-dev", "has_blocked": true }
  ],
  "pagination": {
    "current_page": 1,
    "last_page": 1,
    "per_page": 9,
    "total": 1
  }
}
```

Exemplo: bloquear usuario

```
POST /users/2/block
Accept: application/json

201 Created
{
  "message": "Usuario bloqueado com sucesso.",
  "data": {
    "id": 2,
    "username": "blocked-dev",
    "has_blocked": true
  }
}
```

Efeito persistido: cria registro em **blocks** e remove follows entre os dois usuarios. Efeito visual: o usuario bloqueado sai da busca comum e aparece apenas na lista de bloqueados para desbloqueio.

Qualidade, Testes e Evidencias

```
php artisan test
```

```
Tests: 9 passed (71 assertions)
```

```
npm run build
```

```
vite v7.3.2 building client environment for production...  
600 modules transformed.  
public/build/manifest.json  
public/build/assets/app-CQg5N2Fz.css  
public/build/assets/app-SFA4ad01.js  
built in 1.18s
```

Resposta Direta aos Feedbacks da Fase 02

Ponto criticado	Correcao realizada
Documento generico, sem evidencias concretas	Foram incluidos fluxos reais, endpoints, payloads, status HTTP, services e testes.
Arquitetura pouco conectada ao codigo	A arquitetura agora aponta arquivos reais e explica Pages, Controllers, Requests, Services, Resources e Models.
Diagramas superficiais	Foi adicionada visao em camadas e evolucao arquitetural desde a Fase 01.
Regras de negocio sem demonstracao	Follow, block, busca, perfil e portfolio foram ligados a endpoints, tabelas e testes.
Seções obrigatorias incompletas	Historico, planejamento, requisitos, arquitetura, tecnologias, frontend, backend e integracao foram atualizados.
Evolucao tecnica pouco evidente	A matriz RF01-RF13 mostra o que saiu do planejamento e o que foi implementado ou mantido em backlog.